第10回：音声インタフェースと UX 設計

小野原彩香｜北星学園大学

📌 Zoom録画を後日公開します。課題はメール/Google Driveでも提出可。

参加スタイルについて

この授業は、**多様な参加方法**で学べる設計になっています。

- 演習・全体共有は**チャット書き込み・共有ドキュメント編集**でも参加可
- 発言は任意です。チャットへの書き込みは発言と同等に評価します
- カメラのオン/オフは自由です
- 質問は Zoom チャット・メールのどちらでも受け付けます
- 録画は後日公開します。自分のペースで視聴してください

本日の到達目標

1. 音声 UI の歴史的発展を概観し、現在の多様な利用文脈を把握できる
2. オンデバイス ASR とクラウド ASR のトレードオフを技術・プライバシー両面から説明できる
3. エッジ推論における小型化手法（量子化・プルーニング・知識蒸留）の概念を理解できる
4. 音声インタフェースの UX 設計原則に基づき、アクセシブルなシナリオを提案できる

授業の流れ

時間	内容
0〜10分	前回（第9回）の振り返り・本日の概要説明
10〜45分	スライド講義（音声 UI の歴史〜UX 手法）
45〜65分	演習（UX 設計シナリオ作成）
65〜80分	全体共有
80〜90分	小課題説明・質疑・まとめ

音声 UI の歴史① — IVR 時代（1970年代〜）

IVR（Interactive Voice Response）：音声 UI の起点

- 「1を押すとお問い合わせ、2を押すとご予約」形式
- 音声認識ではなく **DTMF トーン（プッシュ音）** で入力を識別
- 企業の電話窓口・銀行照会・航空券予約に普及
- 問題点：分岐が深くなると迷子になる「**IVR の迷宮**」現象

音声ダイアログシステム（1990年代〜2000年代）

- 限定語彙の音声認識（VoiceXML 規格）
- コールセンターへの導入拡大
- Nuance Communications が市場をリード

> 考えてみよう：IVR と現代の音声アシスタントの最大の違いは何でしょうか？（チャットに書いてみてください）

音声 UI の歴史② — スマートスピーカー以降

製品	年	特徴
Siri（Apple）	2011	スマートフォン統合・自然言語理解
Google Now	2012	検索連携・文脈理解
Alexa（Amazon Echo）	2014	常時起動スピーカー・スマートホーム連携
Google Assistant	2016	マルチターン対話・画像認識統合
ChatGPT 音声モード	2023〜	LLM ベース・自然な感情表現

マルチモーダル UI（現在）

- 音声＋タッチ＋視線＋ジェスチャーの統合
- 車載インフォテインメント、AR/VR ヘッドセット
- 音声＋画面表示の組み合わせ（Echo Show、Google Nest Hub）

UX 設計原則① — エラーリカバリ

音声入力はテキスト入力よりエラーが起きやすい → **回復フローが重要**

> ユーザー：「明日の天気を教えて」
> システム：「明後日の天気ですね、東京は晴れです」（誤認識）
> ユーザー：「違う、明日」
> システム：「失礼しました。明日の天気ですね。
> 東京は曇りのち晴れ、最高気温18度です」

設計のポイント

- 認識結果の読み上げ確認（コンファメーション）
- 短い訂正インタラクション
- 「もう一度おっしゃってください」フレーズの最適化
- 3回以上の失敗で**ヒューマンエスカレーション**

UX 設計原則② — 確認ダイアログとターン設計

確認ダイアログの設計

```
明示的確認（Explicit）：
「○○を削除します。よろしいですか？」
→ 重要・取り消し不可操作に使用

暗黙的確認（Implicit）：
「承知しました、山田さんに電話します」
→ 低リスク操作に使用
```

ターン設計の原則

原則	理由
簡潔な応答	音声は読み返せない → 重要情報は前半に
選択肢の制限（最大2択）	3択以上は認知負荷が高い
プロンプトの明確化	「どうぞ」より「何を検索しますか？」
沈黙のハンドリング	適切なタイムアウトと再プロンプト

クラウド ASR vs オンデバイス ASR

【クラウドASR】
音声データ → インターネット → クラウドサーバー（巨大モデル） → テキスト → 端末

【オンデバイスASR】
音声データ → 端末内プロセッサ（小型モデル） → テキスト（外部送信なし）

指標	クラウド ASR	オンデバイス ASR
往復遅延	200ms〜1000ms（回線依存）	50ms〜200ms（端末性能依存）
オフライン動作	不可	可
認識精度	高（最新の大規模モデル）	中〜高（モデル規模依存）
対応言語数	多い（100言語超）	少ない（主要言語）

プライバシーとセキュリティ

クラウド ASR のリスク

- 音声データが外部サーバーに送信される
- 「常時マイクオン」デバイスでの意図しない録音リスク
- サービス提供企業による学習利用（契約内容次第）

実際の事例

- 2019年：Google・Amazon が音声クリップを人間に聴取させていたことが発覚
- 2023年：各社がプライバシーポリシーを改定・オプトアウト機能を強化

オンデバイスASR のプライバシー利点

- 音声データが端末外に出ない
- GDPR・個人情報保護法との相性が良い
- 医療・法律・金融分野での利用に適合

エッジ推論① — 量子化（Quantization）

概念：モデルのパラメータを高精度から低精度に変換

```
float32 : 1パラメータ = 32ビット（4バイト）
int8    : 1パラメータ =  8ビット（1バイト）→ メモリ 1/4
int4    : 1パラメータ =  4ビット（0.5バイト）→ メモリ 1/8
```

トレードオフ

- メモリ削減・推論高速化 ↔️ 精度のわずかな低下
- Post-Training Quantization（PTQ）vs. Quantization-Aware Training（QAT）

エッジ推論② — プルーニングと知識蒸留

プルーニング（Pruning）：重要度の低いパラメータを削除

```
密なモデル（Dense）→ 疎なモデル（Sparse）
100億パラメータ    → 重要な 30億パラメータのみ残す
```

- 非構造化プルーニング：個々の重みを削除（スパース行列）
- 構造化プルーニング：チャネル・レイヤー全体を削除（ハードウェア最適化しやすい）

知識蒸留（Knowledge Distillation）：大規模モデルの知識を小規模モデルに転移

```
教師モデル（Whisper Large-V3：15億パラメータ）
    ↓ 教師の出力確率分布（Soft Labels）を使って訓練
生徒モデル（Whisper Tiny：3900万パラメータ）
```

> なぜ有効か？ 正解ラベル（One-hot）だけでなく、教師の「不確かさ」も学習できる。

オンデバイス ASR の代表例

Whisper.cpp（CPU 動作）

項目	内容
対応プラットフォーム	Mac, Linux, Windows, Raspberry Pi, iOS, Android
量子化対応	4-bit 量子化でメモリ大幅削減
ライセンス	MIT（商用利用可）
Raspberry Pi 4 での速度	リアルタイムの約 2〜4 倍速（tiny モデル）

Vosk（Kaldi ベース）

項目	内容
モデルサイズ	50MB〜2GB
日本語対応	vosk-model-ja として公開
強み	組み込みシステム統合・カスタム語彙追加が容易

Apple Siri on Neural Engine：A17 Pro で毎秒 35 兆回の演算（35 TOPS）、2021〜デフォルトでオンデバイス化

音声 UI のアクセシビリティ設計

音声インタフェースはアクセシビリティを高める一方で、**新たなバリアを生む可能性**もある。

発話困難の原因（多様）

- 吃音（Stuttering）
- 失語症（Aphasia）・構音障害（Dysarthria）
- 喉頭摘出後
- 緊張・不安による一時的な発話困難
- 場所的制約（図書館・電車内等）

設計上の対策

課題	対策
吃音による誤認識	タイムアウトを長めに設定・繰り返し発話への対応
声量が小さい	感度の自動調整・増幅機能
発話自体が困難	テキスト入力・スイッチアクセスをフォールバックとして**常時**提供

Google Project Euphonia と高齢者対応

Google Project Euphonia

- 非標準的な音声（ALS 患者・脳性麻痺者等）の認識精度向上プロジェクト
- ユーザーの音声サンプルで個別モデルをファインチューニング
- 精度改善の事例：発話明瞭度 32% → **90%以上**

高齢者の音声特性と設計上の対策

特性	対策
声帯の変化・発話速度の低下	読み上げ速度の調整機能（低速モード）
繰り返し・言い直しが多い	許容度の向上
高音域の聴覚低下	大音量・視覚フィードバック
複雑なコマンド構文	シンプルな設計

> 「障害者のために作られた設計は、すべての人にとって良い設計になる」――ユニバーサルデザインの原則

UX リサーチ手法

Wizard of Oz（WoZ）法：人間がシステムの応答を模倣し、プロトタイプなしでユーザーテストを行う

```
参加者（ユーザー役）  ←音声→  マイク
                                  ↓
                        ［別室のオペレーター］（応答を模倣）
                                  ↓
参加者                ←スピーカー←  応答
```

なぜ有効か？ 技術的制約に縛られない理想のシナリオを早期段階で探索できる。

その他の手法

手法	音声 UI での活用
Think Aloud 法	エラー時の思考プロセス把握
発話ログ分析	頻出コマンド・失敗パターン発見
A/B テスト	最適プロンプト文の選定
ヒューリスティック評価	設計初期の問題発見

音声 UI のユーザビリティ評価指標

指標	定義
Task Completion Rate	タスクを完了できたユーザーの割合
Turn Count	タスク完了に要した対話ターン数
WER（Word Error Rate）	音声認識誤り率（第9回で学習）
Time on Task	タスク完了に要した時間
Recovery Rate	エラーから回復した割合
User Satisfaction（SUS 等）	ユーザー主観評価

> WER は低ければ良いが、WER が低くてもタスク完了率が低い場合は UX の問題。
> → 技術的な指標とユーザー体験指標を組み合わせて評価する。

演習：音声インタフェースを UX 視点で設計する（20分）

目標：UX 設計原則とアクセシビリティの知識を活かし、具体的なシナリオを作成する。
自分のペースで取り組んでください。Zoom 参加またはメール提出でも可。

STEP 1：ペルソナの決定（5分）

```
ペルソナ名（仮名でOK）：
年齢・職業：
音声UI利用シーン：
使用デバイス：
特別な配慮が必要な点（あれば）：
```

ペルソナ例のヒント：

- 視覚障害のある 30 代会社員が通勤中に使う音声メモアプリ
- 80 代の祖父母が使う薬の服用管理アシスタント
- 吃音のある大学生が講義中に使う静音テキスト変換ツール
- 調理中の料理人向けレシピナビ

演習：シナリオ作成（STEP 2〜3）

STEP 2：シナリオの作成（10分）

> 【場面設定】
> （どこで・何をしているときに音声UIを使うか）
>
> 【タスク】
> （達成したいこと）
>
> 【対話シナリオ（3〜5ターン）】
> ユーザー：「〇〇」
> システム：「〇〇」
> ユーザー：「〇〇」
> システム：「〇〇」
>
> 【エラーケース（1つ）】
> （誤認識が起きたとき、どう回復するか）
>
> 【アクセシビリティ配慮】
> （どんな多様なユーザーに対応しているか）

STEP 3：共有（5分）

チャットまたは共有ドキュメントにシナリオを貼り付けてください。
Zoom 参加中の方は口頭での紹介も歓迎。

全体共有（15分）

投稿いただいたシナリオへの**講評のポイント**：

1. エラーリカバリは考慮されているか？
2. アクセシビリティへの配慮はあるか？
3. 第8〜9回までの技術（Whisper・CER等）で実現できる設計か？

小課題

提出方法：Google Drive または メール添付
締め切り：翌週末 23:59
形式：400字程度（超えても問題ありません）
備考：音声録音での提出・箇条書きでの提出も可。分量より内容を重視します。

課題内容：

> 音声インタフェースの**良い点・問題点をそれぞれ3つ**挙げ、UX 改善案を提案せよ。

解答の観点（必ずしも従う必要はありません）：

- **技術的改善**（ASR 精度向上・ノイズ除去等）
- **デザイン的改善**（UI 設計・フィードバック方法等）
- **社会的・倫理的改善**（データ利用ポリシー・アクセシビリティ等）

本日のまとめ

1. **音声 UI の歴史**：IVR（DTMF）→ 限定語彙 ASR → スマートアシスタント → LLM ベース音声 UI へ進化。
2. **UX 設計原則**：エラーリカバリ・確認ダイアログ・ターン設計・マルチターン対話の文脈保持。
3. **クラウド vs オンデバイス**：精度・遅延・プライバシー・オフライン動作のトレードオフ。
4. **エッジ推論の小型化**：量子化・プルーニング・知識蒸留の3手法。
5. **アクセシビリティ設計**：多様な発話困難者・高齢者への配慮。フォールバック手段を常時提供。
6. **UX リサーチ**：WoZ 法・Think Aloud・発話ログ分析など。技術指標（WER）とユーザー体験指標を組み合わせる。

次回予告（第11回：動画回）

自然言語処理との接続（NLU と音声）

- 音声処理パイプライン全体（ASR → NLU → DM → NLG → TTS）
- 意図推定・スロット抽出・NER
- LLM との接続（Whisper + ChatGPT）
- Python 実習（Google Colab）

事前準備（任意）：Google Colab で Whisper を動かしてみる：

```
!pip install openai-whisper
import whisper
model = whisper.load_model("tiny")
```